



TRƯỜNG ĐẠI HỌC KINH TẾ KỸ THUẬT CÔNG NGHIỆP
Khoa Công Nghệ Thông Tin
----------

LẬP TRÌNH DI ĐỘNG



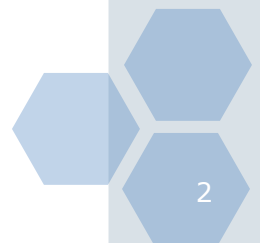
CHƯƠNG 3: LẬP TRÌNH USER INTERFACE TRONG ANDROID STUDIO





Chương 1 – Tổng quan về lập trình Android

-  **1.1 THÀNH PHẦN GIAO DIỆN TRONG ANDROID**
-  **1.2 XÂY DỰNG GIAO DIỆN SỬ DỤNG XML**
-  **1.3 MỘT SỐ LAYOUT THÔNG DỤNG**
-  **1.4 MỘT SỐ VIEW THÔNG DỤNG**





UNETI



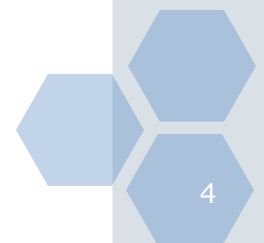
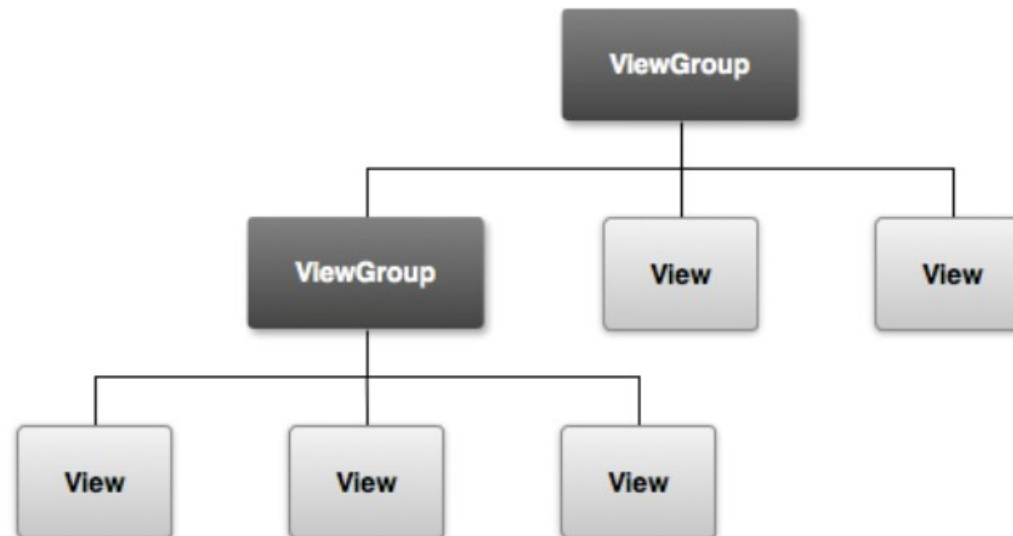
1.1 THÀNH PHẦN TRONG GIAO DIỆN ANDROID





1.1 THÀNH PHẦN TRONG GIAO DIỆN ANDROID

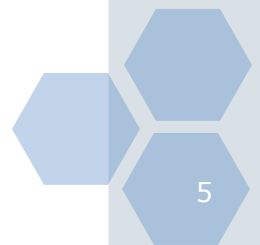
- View – Widget
- ViewGroup – Layout





1.1.1 VIEW - WIDGET

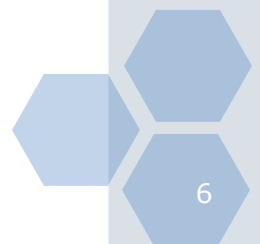
- **View** là đối tượng cơ bản để xây dựng mọi thành phần của giao diện đồ họa
- Hầu hết các thành phần cơ bản của giao diện đều kế thừa từ View: **TextView**, **Button**, **Spinner**, **ToggleButton**, **RadioButton**,...
- Các thành phần này hầu hết đều nằm trong gói **android.widget** nên thường gọi là **widget**
- Lập trình viên có thể tự tạo **widget** của riêng mình bằng cách tùy biến view theo cách của riêng mình





1.1.2 VIEWGROUP - LAYOUT

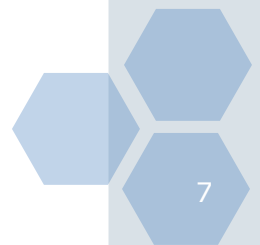
- **ViewGroup** là các view đặc biệt, có thể chứa bên trong nó các view khác
 - VD1: trong một bài báo gồm các textview, imageview,...
 - VD2: một thư viện ảnh gồm các listview, imageview,...
- ViewGroup là cửa sổ cha của các view con
- Một view nằm trong ViewGroup cần phải có thông tin về vị trí của nó trong cửa sổ cha
- ViewGroup = các view con + cách bố trí các view con đó bên trong





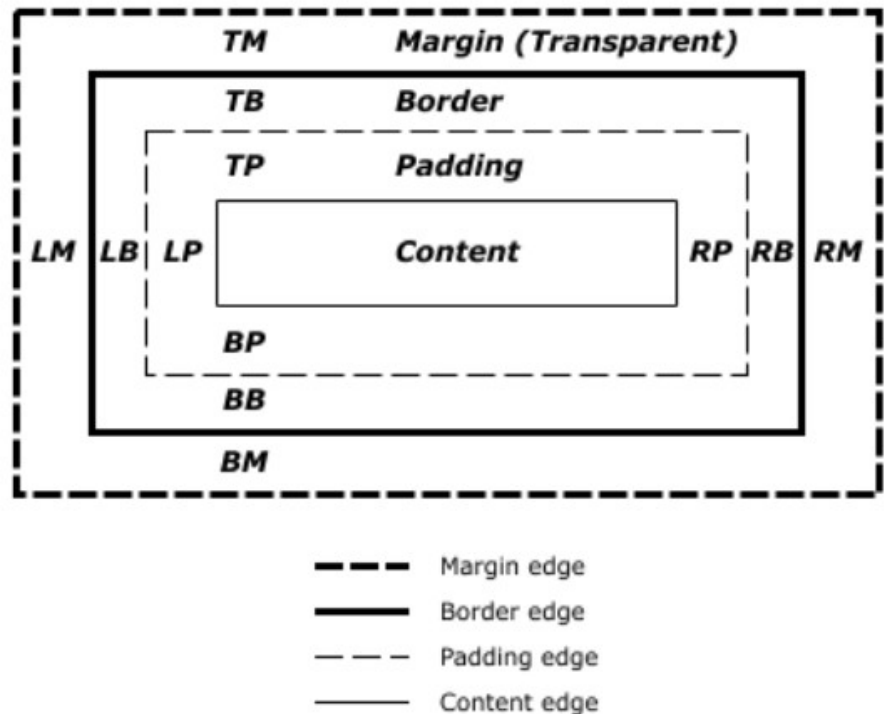
1.1.3 THAM SỐ CỦA LAYOUT VÀ VIEW

- Bản thân layout và view cũng có các tham số của nó khi được đặt vào view cha
 - **Vị trí** (position): cặp tọa độ Left/Top
 - **Kích thước** (size): cặp giá trị Width/Height
 - **Lề** (margin): tham số trong LayoutParams (kiểu MarginLayoutParams), quy định khoảng cách của view với các thành phần xung quanh
 - **Đệm** (padding): vùng trống từ nội dung của view ra các viền, sử dụng phương thức `setPadding(int,int,int,int)` để điều chỉnh, đơn vị đo thường là dp



1.1.3 THAM SỐ CỦA LAYOUT VÀ VIEW

- Kích thước của view không bao gồm độ dày của margin
- Trong android không có khái niệm border
- Muốn một view có border, lập trình viên sử dụng thủ thuật thiết lập border thông qua background





UNETI



1.2 XÂY DỰNG GIAO DIỆN SỬ DỤNG XML





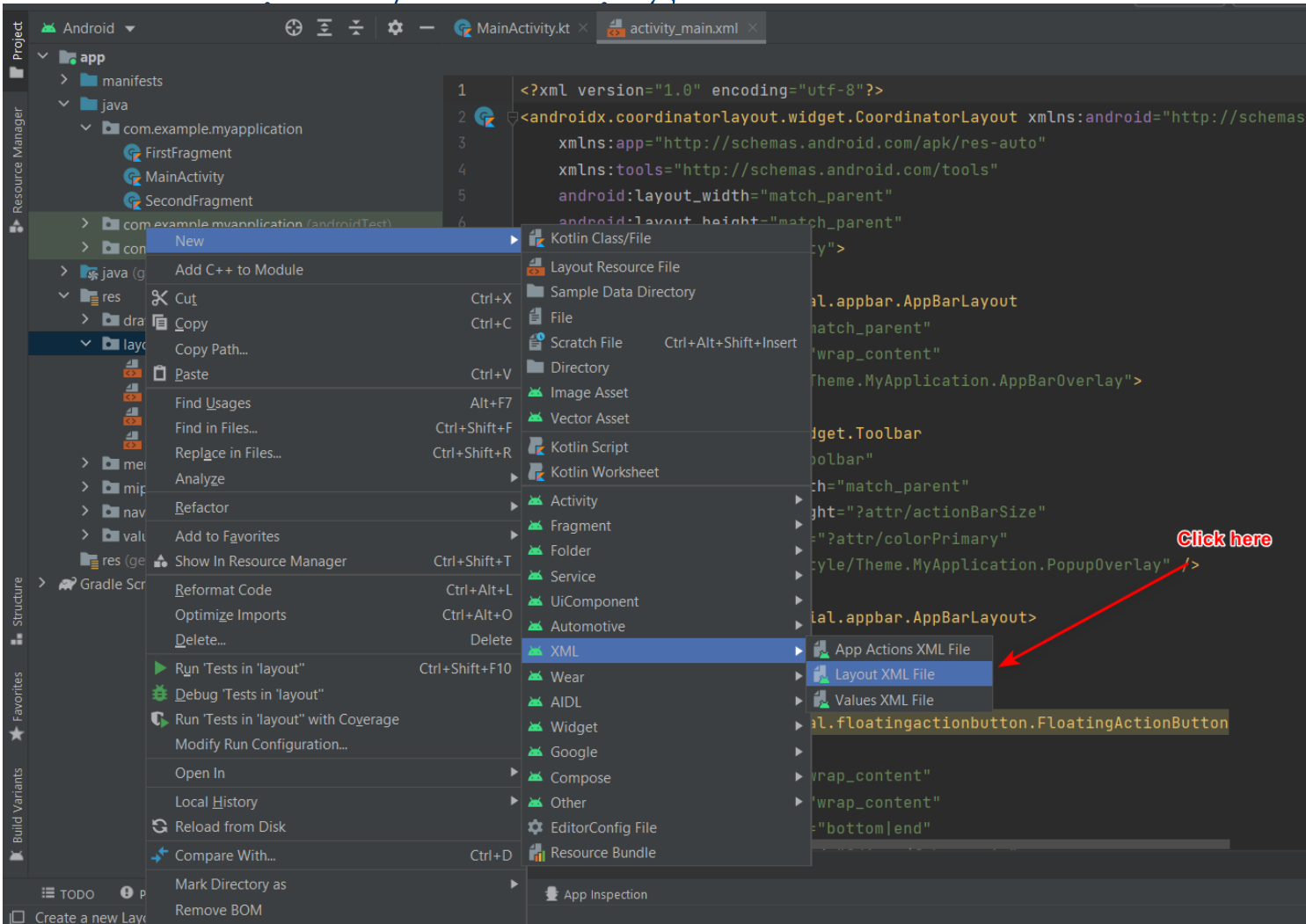
1.2 XÂY DỰNG GIAO DIỆN SỬ DỤNG XML

- XML là phương pháp tạo giao diện phổ biến nhất
- XML có cấu trúc dễ hiểu, phân cấp, giống HTML
- Tên của thành phần XML tương ứng với lớp java trong code
- Dễ chỉnh sửa trên bằng design hoặc sửa file XML

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout xmlns:android="http://.../res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    android:orientation="vertical" >
    <Button android:id="@+id/button"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, I am a Button" />
    <TextView android:id="@+id/text"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Hello, I am a TextView" />
</LinearLayout>
```



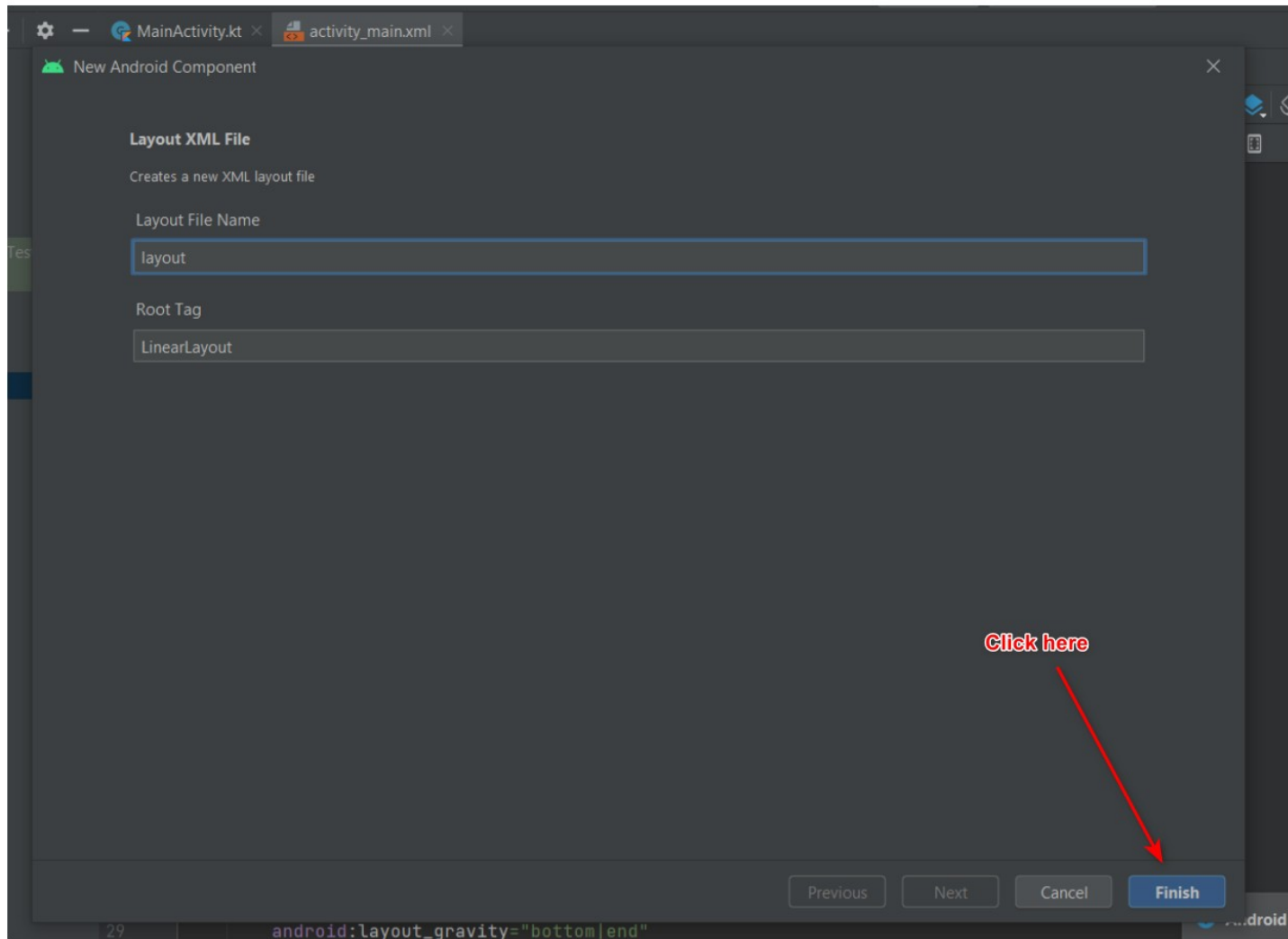
- Các bước tạo Layout sử dụng XML:





1.2 XÂY DỰNG GIAO DIỆN SỬ DỤNG XML

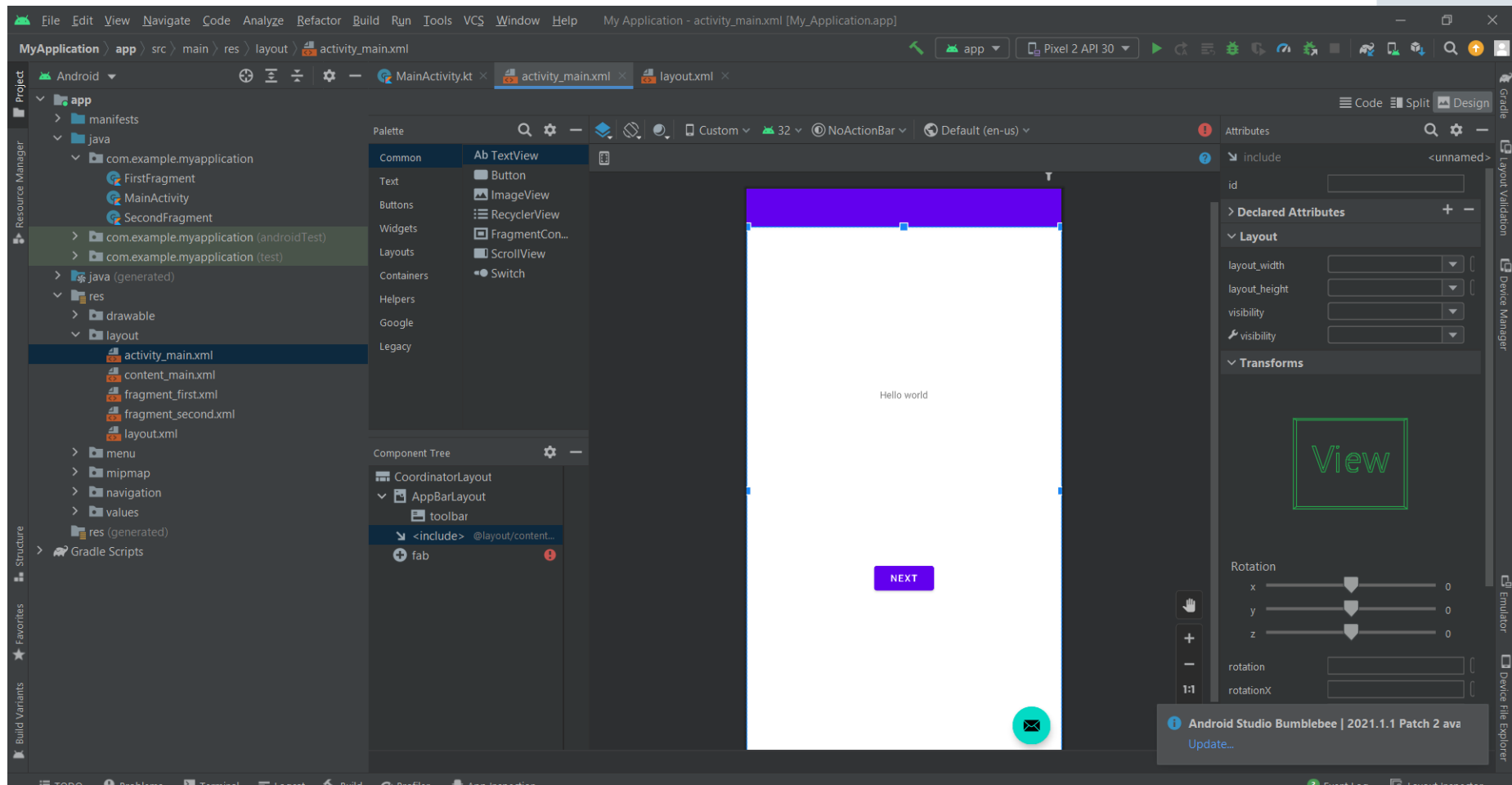
- Các bước tạo Layout sử dụng XML





1.2 XÂY DỰNG GIAO DIỆN SỬ DỤNG XML

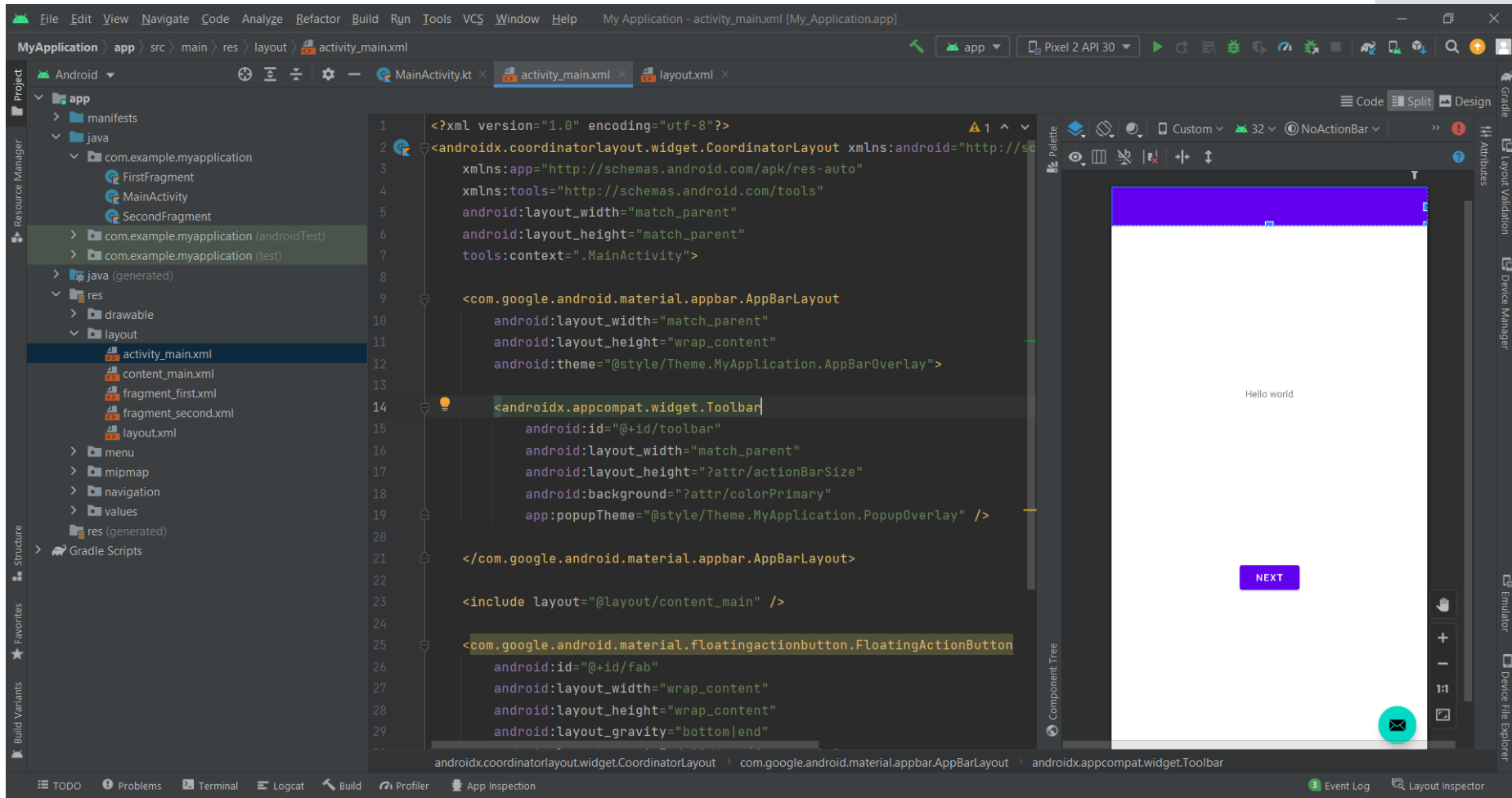
- Sử dụng giao diện kéo thả của Android Studio





1.2 XÂY DỰNG GIAO DIỆN SỬ DỤNG XML

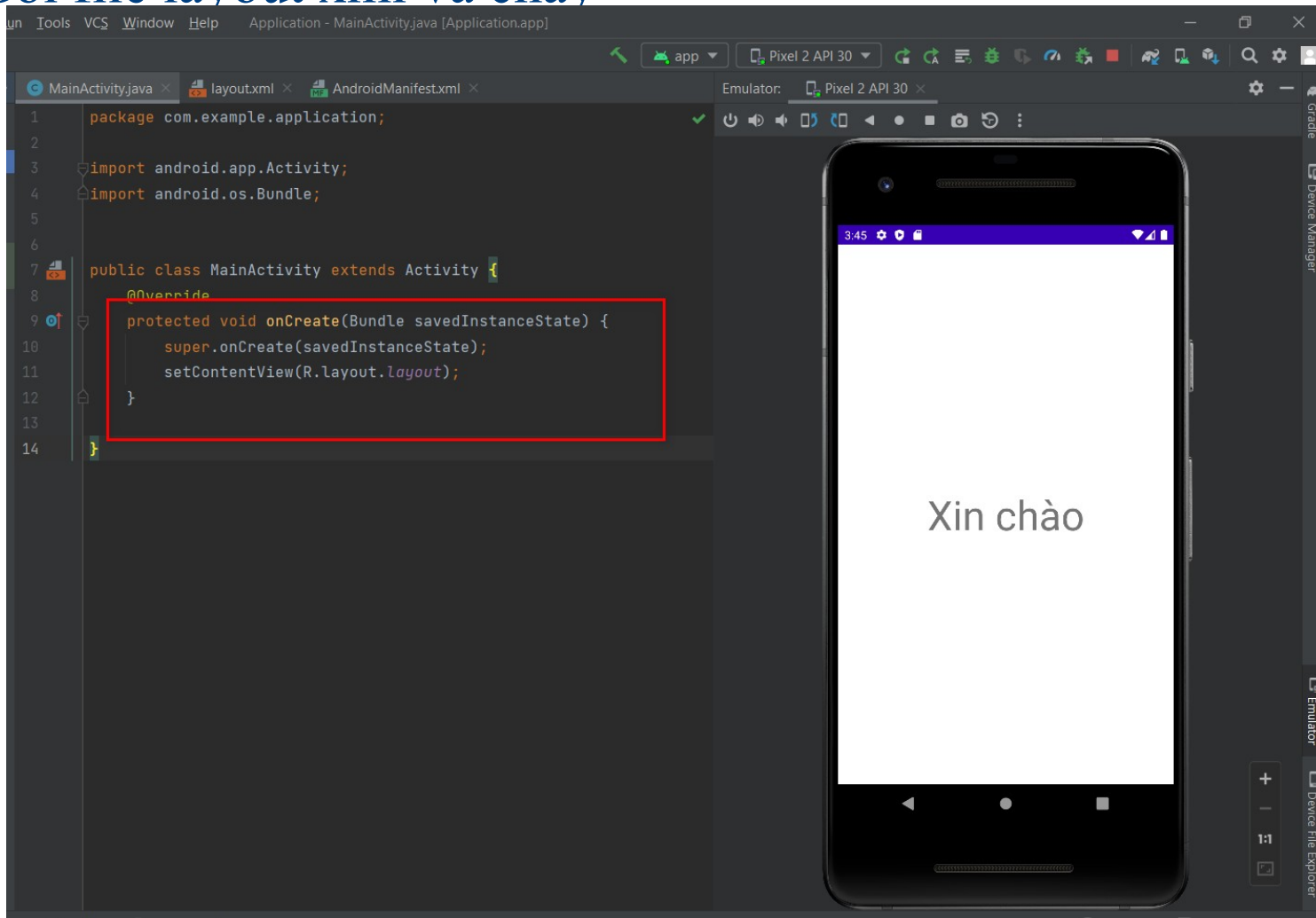
- Code trực tiếp trên file xml





1.2 XÂY DỰNG GIAO DIỆN SỬ DỤNG XML

- Goi file layout xml và chạy





UNETI



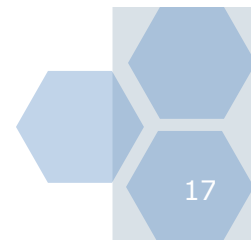
1.3 MỘT SỐ LAYOUT THÔNG DỤNG





1.3.1 LinearLayout

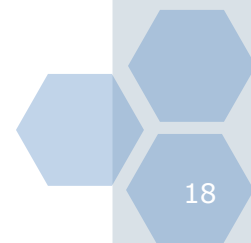
```
<?xmlversion="1.0"encoding="utf-8"?>
<LinearLayoutxmlns:android="http://schemas.android.com/apk/res/a
ndroid"
android:orientation="vertical"
android:layout_width="fill_parent"
android:layout_height="fill_parent"
>
    <TextView
        android:layout_width="fill_parent"
        android:layout_height="wrap_content"
        android:text="@string/hello"
    />
</LinearLayout>
```





1.3.1 LinearLayout

- Các view bên trong nó được xếp liên tiếp thành một hàng hoặc một cột
- Thuộc tính `android:orientation` quy định cách bố cục theo hàng hay theo cột, giá trị có thể là:
 - “vertical”: các view bên trong LinearLayout được sắp xếp theo chiều dọc
 - “horizontal”: các view bên trong sắp xếp theo chiều ngang
- LinearLayout không thay đổi kích thước các view con, chỉ điều chỉnh vị trí của chúng





1.3.1 LinearLayout

LinearLayout Demo

Back

First Name

Last Name

HelloHe

Username Nickname Back



1.3.2 RelativeLayout

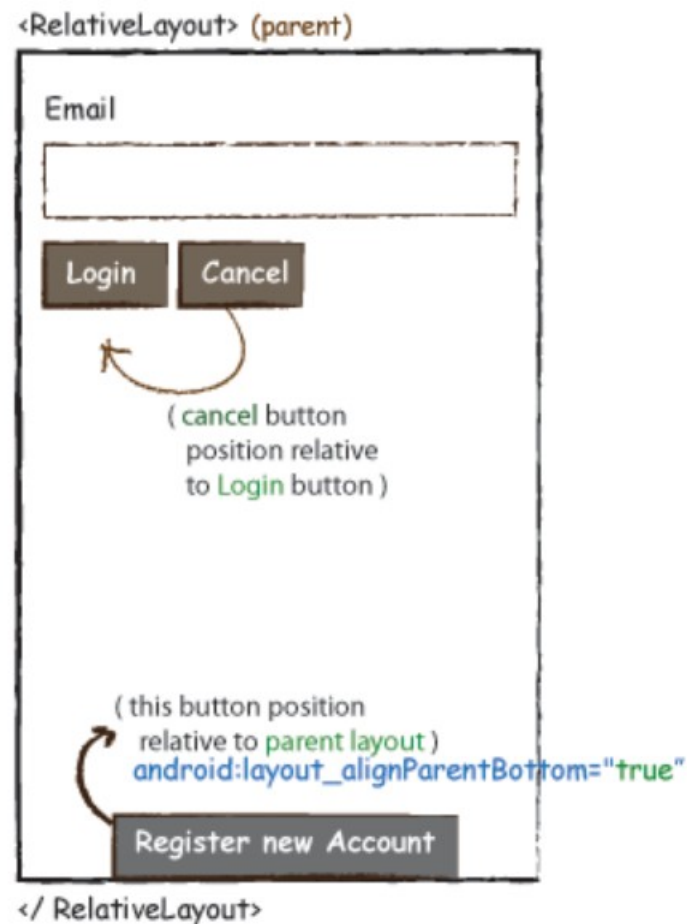
- Các view con trong layout xác định vị trí và kích thước dựa trên quan hệ với view cha hoặc các view con khác
- Dùng trong trường hợp đặt trọng tâm vào mối quan hệ giữa các thành phần

```
<?xmlversion="1.0"encoding="utf-8"?>
<RelativeLayout
    android:id="@+id/RLLayout"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    xmlns:android="http://schemas.android.com/apk/res/android"
>
    <TextView
        android:id="@+id/lblComments"
        android:layout_width="wrap_content"
        android:layout_height="wrap_content"
        android:text="Comments"
        android:layout_alignParentTop="true"
        android:layout_alignParentLeft="true"
    />
    <EditText
        android:id="@+id/txtComments"
        android:layout_width="fill_parent"
        android:layout_height="170px"
        android:textSize="18sp"
        android:layout_alignLeft="@+id/lblComments"
        android:layout_below="@+id/lblComments"
        android:layout_centerHorizontal="true"
    />
    <Button
        android:id="@+id/btnSave"
        android:layout_width="125px"
        android:layout_height="wrap_content"
        android:text="Save"
        android:layout_below="@+id/txtComments"
        android:layout_alignRight="@+id/txtComments"
    />
    <Button
        android:id="@+id/btnCancel"
        android:layout_width="124px"
        android:layout_height="wrap_content"
        android:text="Cancel"
        android:layout_below="@+id/txtComments"
        android:layout_alignLeft="@+id/txtComments"
    />
</RelativeLayout> |
```



1.3.2 RelativeLayout

```
<TextView android:id="@+id/label"
android:layout_width="fill_parent"
android:layout_height="wrap_content"
android:text="Email" />
<EditText android:id="@+id/inputEmail"
android:layout_width="fill_parent"
android:layout_height="wrap_content"
android:layout_below="@id/label" />
<Button
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_alignParentBottom="true"
android:text="Register new Account"
android:layout_centerHorizontal="true"/>
```

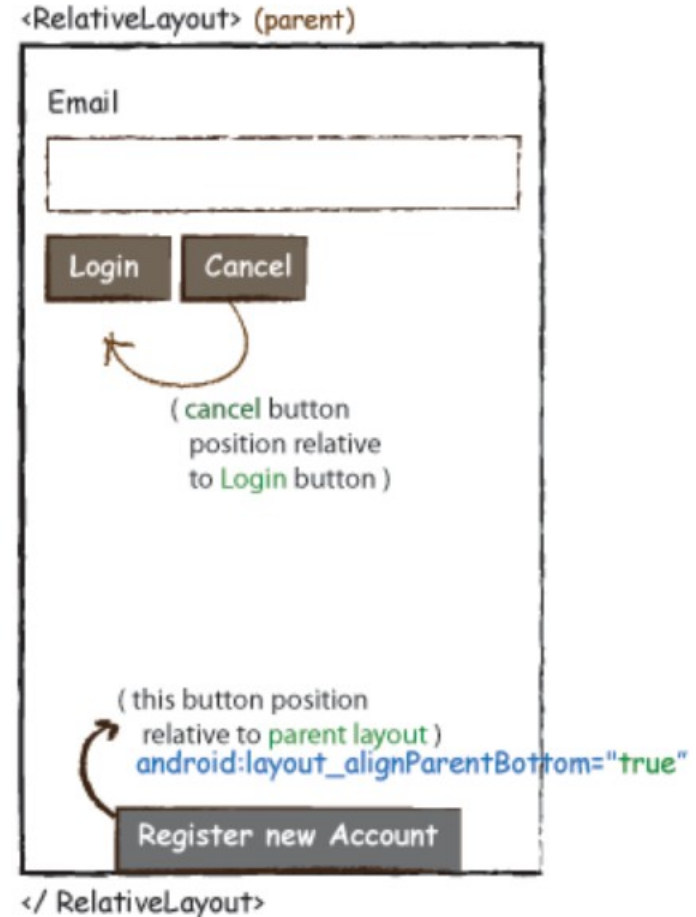




1.3.2 RelativeLayout

```
<Button android:id="@+id/btnLogin"
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_below="@id/inputEmail"
android:layout_alignParentLeft="true"
android:layout_marginRight="10px"
android:text="Login" />
```

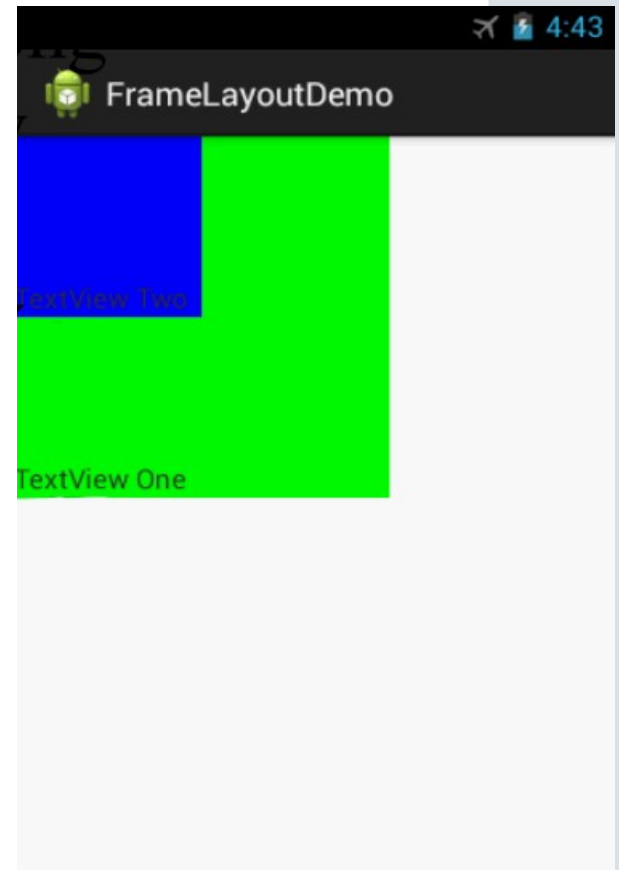
```
<Button
android:layout_width="wrap_content"
android:layout_height="wrap_content"
android:layout_toRightOf="@id/btnLogin"
android:layout_alignTop="@id/btnLogin"
android:text="Cancel" />
```



1.3.3. FrameLayout

- Các view con được đặt liên tiếp chồng lên nhau, view sau đặt lên trên view trước
- Layout không tự động đổi kích thước của view con
- Có thể chuyển view con lên trên bằng code:

```
parent.bringChildToFront(child);  
parent.invalidate();
```





1.3.3. FrameLayout

- **ScrollView & HorizontalScrollView**
 - ScrollView và HorizontalScrollView là trường hợp đặc biệt của FrameLayout
 - Cho phép view con có thể có kích thước lớn hơn view cha
 - Trong trường hợp view con nhỏ hơn view cha, người dùng chỉ nhìn và tương tác với view con
 - Trường hợp view con có kích thước lớn hơn view cha, ScrollView và HorizontalScrollView sẽ tự động xuất hiện các thanh cuộn phù hợp





1.3.4 TableLayout

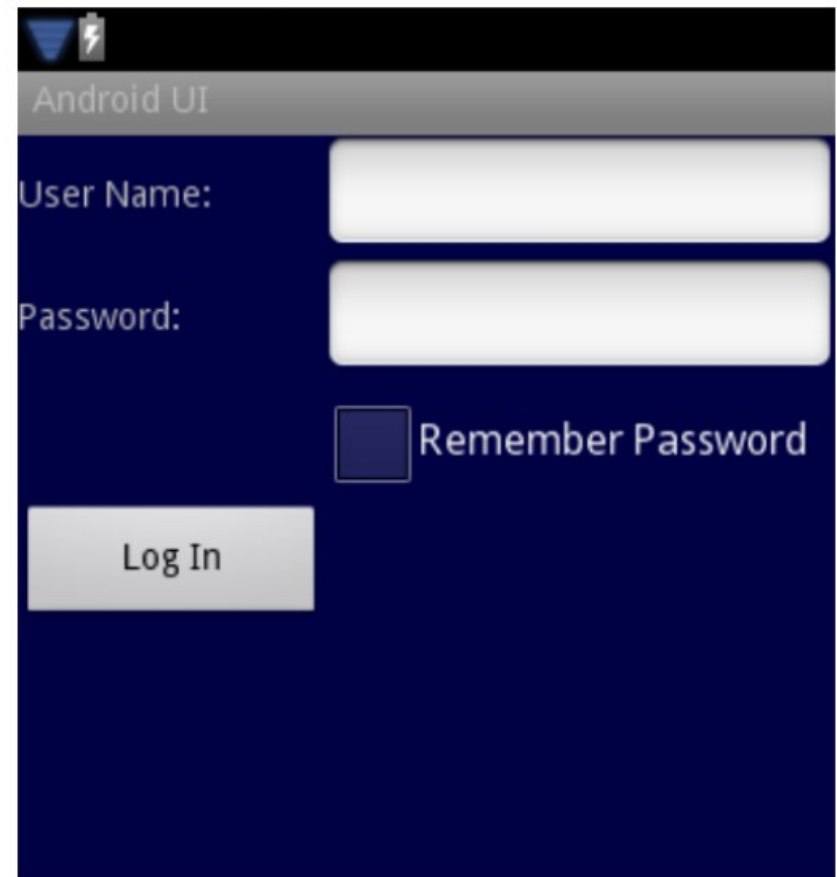
- TableLayout dùng để tổ chức các đối tượng view dưới dạng một bảng gồm nhiều dòng và cột
- Mỗi dòng nằm trong một thẻ <TableRow>
- Mỗi đối tượng view đặt trên một dòng sẽ tạo thành một ô trong giao diện lưới do TableLayout tạo ra
- Chiều rộng của mỗi cột được xác định bằng chiều rộng lớn nhất của các ô nằm trên cùng một cột
- Kích thước của mỗi dòng cột không nhất thiết phải bằng nhau





1.3.4 TableLayout

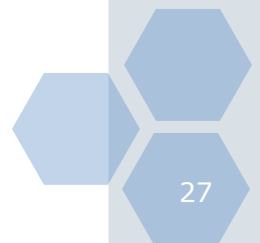
```
<?xmlversion="1.0"encoding="utf-8"?>
<TableLayout
  xmlns:android="http://schemas.android.com/apk/res/android"
  android:layout_height="fill_parent"
  android:layout_width="fill_parent"
  android:background="#000044">
  <TableRow>
    <TextView android:text="User Name:"
      android:width="120px"
    />
    <EditText android:id="@+id/txtUserName"
      android:width="200px"/>
  </TableRow>
  <TableRow>
    <TextView android:text="Password:"
    />
    <EditText android:id="@+id/txtPassword"
      android:password="true"
    />
  </TableRow>
  <TableRow>
    <TextView/>
    <CheckBox android:id="@+id/chkRememberPassword"
      android:layout_width="fill_parent"
      android:layout_height="wrap_content"
      android:text="Remember Password"
    />
  </TableRow>
  <TableRow>
    <Button android:id="@+id/buttonSignIn"
      android:text="Log In"/>
  </TableRow>
</TableLayout>
```





1.3.4 TableLayout

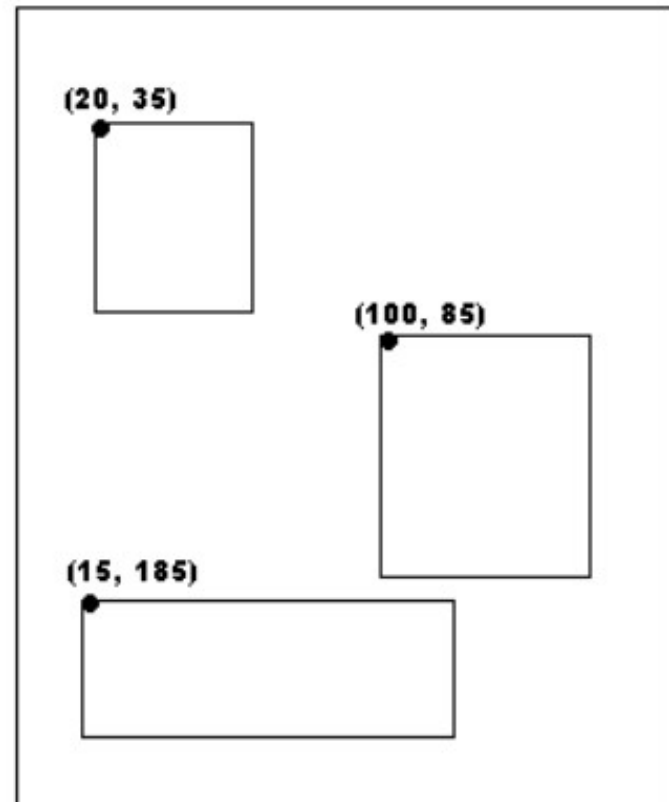
- Thông thường mỗi view sẽ chiếm một ô trên lưới
- Trường hợp muốn để trống ô ta có thể đặt vào đó một textview trống (bằng thẻ `<TextView />`)
- Tuy nhiên ta cũng có thể chỉ định kích thước và vị trí của view thông qua các thuộc tính ẩn:
 - “android:layout_span”: cỡ của view (bao nhiêu cột)
 - “android:layout_column”: vị trí cột đặt view



1.3.5 AbsoluteLayout

- Absolute Layout trong Android giúp bạn xác định vị trí chính xác (các tọa độ x/y) của view con.
- Absolute Layout là ít linh động và khó duy trì hơn các loại Layout mà không có vị trí tuyệt đối khác.

Absolute Layout





1.3.5 AbsoluteLayout

```
<AbsoluteLayout xmlns:android="http://schemas.android.com/apk/res/android"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent">

    <Button
        android:layout_width="100dp"
        android:layout_height="wrap_content"
        android:text="OK"
        android:layout_x="50px"
        android:layout_y="361px" />

    <Button
        android:layout_width="100dp"
        android:layout_height="wrap_content"
        android:text="Cancel"
        android:layout_x="225px"
        android:layout_y="361px" />

</AbsoluteLayout>
```





UNETI



1.4 MỘT SỐ VIEW THÔNG DỤNG

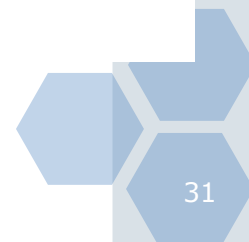




1.4.1 TextView

- Mục đích: hiển thị văn bản
- Một số thuộc tính hay được sử dụng:
 - android:layout_width
 - android:layout_height
 - android:text
 - android:textColor
 - android:textSize
 - android:gravity

```
<TextView  
    Android:layout_width="fill_parent"  
    Android:layout_height="wrap_content"  
    Android:text="Hello World! Demo TextView"  
    Android:textColor="#07a931"  
    Android:textSize="20px"  
    Android:gravity="center_horizontal"  
>
```

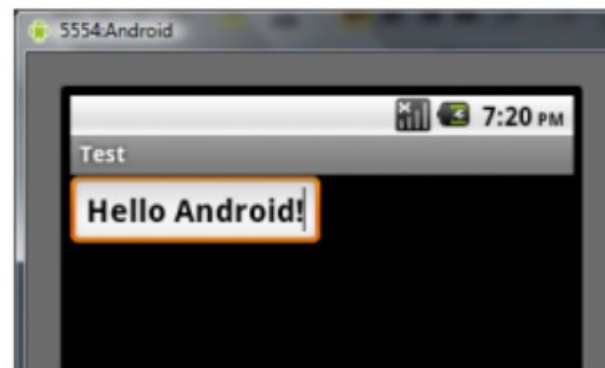


1.4.2 EditText

- Mục đích: hiển thị và cho phép nhập dữ liệu
- Chú ý:
 - Thuộc tính “**android:singleLine**” bằng false thì EditText sẽ là một Textbox, ngược lại nó là một TextField (cho phép nhập liệu nhiều dòng)
 - Lấy nội dung: **editText.getText().toString()**

<EditText

```
Android:id="@+id/EditText01"  
Android:layout_width="wrap_content"  
Android:layout_height="wrap_content"  
Android:textStyle="bold"  
Android:textSize="20dip"  
Android:textColor="#000000"  
Android:text="Hello Android!"  
Android:singleLine="true"  
Android:inputType="textCapWords"  
>
```





1.4.2 Button & ImageButton

- Mục đích: nhận lệnh bấm từ người dùng
- Thuộc tính “**android:onClick**” chỉ ra phương thức sẽ khởi chạy khi nút được bấm
- ImageButton sử dụng hình ảnh thay vì text

<Button

```
    Android:layout_width="wrap_content"  
    Android:layout_height="wrap_content"  
    Android:id="@+id/cmdButton1"  
    Android:text="Touch me!"
```

/>

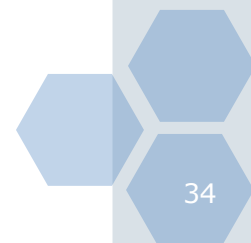
```
<ImageButton android:id="@+id/btnImg1"  
    android:layout_width="fill_parent"  
    android:layout_height="wrap_content"  
    android:src="@drawable/icon"  
/>
```





1.4.3 ToggleButton

- Mục đích: hiển thị trạng thái on/off
- Thuộc tính quan trọng
 - “`android:textOn`”: text hiện ở trạng thái on
 - “`android:textOff`”: text hiện ở trạng thái off
 - “`android:onClick`”: tương tự như ở Button
- Phương thức “`bool isChecked()`” trả về trạng thái on/off
- Từ phiên bản 4.0, android có điều khiển Switch tương tự như ToggleButton nhưng khác về cách hiển thị thông tin





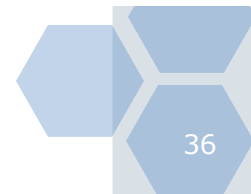
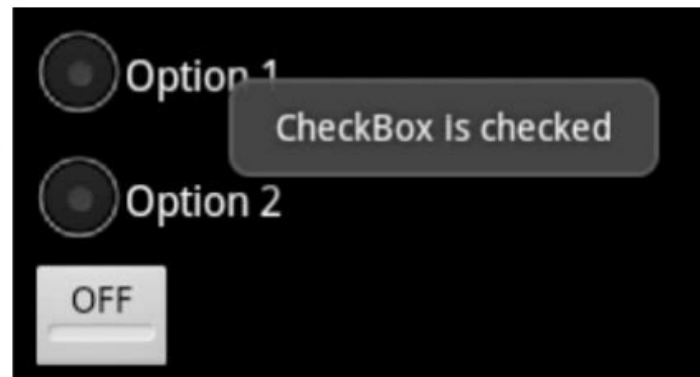
1.4.4 CheckBox

- Mục đích: đưa ra một ô check để người dùng có thể xác nhận có lựa chọn hay không
- Thuộc tính quan trọng
 - “`android:checked`”: thiết lập trạng thái ban đầu
 - “`android:text`”: nội dung đi kèm với check box
 - “`android:onClick`”: tương tự như ở Button
- Phương thức “`bool isChecked()`” trả về trạng thái on/off
- Phương thức “`void setChecked(bool)`” để thiết lập trạng thái on/off



1.4.5 RadioGroup & RadioButton

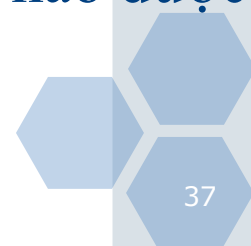
```
<RadioGroup android:id="@+id/rdbGp1"
  android:layout_width="fill_parent"
  android:layout_height="wrap_content"
  android:orientation="vertical"
>
<RadioButton android:id="@+id/rdb1"
  android:layout_width="fill_parent"
  android:layout_height="wrap_content"
  android:text="Option 1"
/>
<RadioButton android:id="@+id/rdb2"
  android:layout_width="fill_parent"
  android:layout_height="wrap_content"
  android:text="Option 2"
/>
</RadioGroup>
```





1.4.5 RadioGroup & RadioButton

- RadioGroup kế thừa từ LinearLayout (mặc định là căn theo cột – vertical)
- RadioButton là các view cho phép lựa chọn on/off (dùng “`boolean isChecked()`” để kiểm tra)
- RadioGroup đảm bảo chỉ tối đa một RadioButton được chọn vào một thời điểm
- Từ RadioGroup để lấy ID của RadioButton đang bật, dùng hàm “`int getCheckedRadioButtonId()`” (nếu không có RadioButton nào được lựa chọn thì hàm trả về -1)

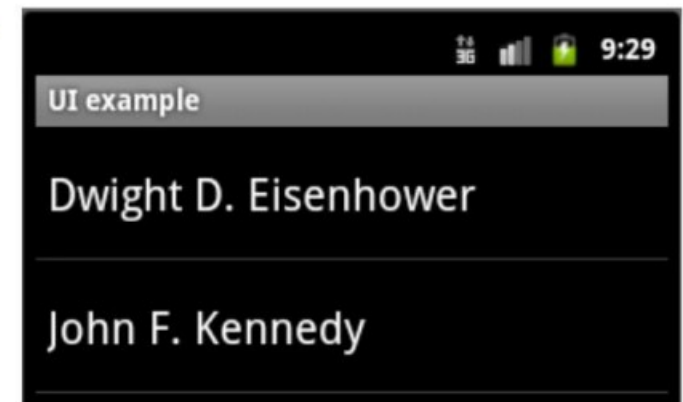




1.4.6 ListView

- ListView hiển thị một danh sách các phần tử trên một giao diện cho phép cuộn theo chiều dọc

```
<?xml version="1.0" encoding="utf-8"?>
<LinearLayout
    xmlns:android="http://schemas.android.com/apk/res/android"
    android:orientation="vertical"
    android:layout_width="fill_parent"
    android:layout_height="fill_parent"
    >
    <ListView android:id="@+id/android:list"
        android:layout_width="fill_parent"
        android:layout_height="fill_parent"/>
</LinearLayout>
```





1.4.6 ListView

- Thiết lập nội dung cho ListView

// mỗi string là 1 dòng trong ListView

```
String[] values = new String[] {  
    "Việt Nam", "Trung Quốc", "Triều Tiên", "Cuba", "Hoa Kỳ"};
```

// Adapter: chứa dữ liệu và view của 1 dòng

// 1. Context

// 2. Layout cho 1 dòng

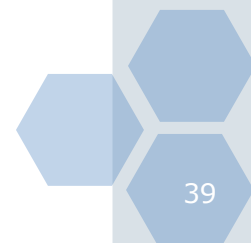
// 3. ID của TextView để hiện dữ liệu

// 4. Mảng các dữ liệu

```
ArrayAdapter<String> adapter = new ArrayAdapter<String>(  
    this, android.R.layout.simple_list_item_1,  
    android.R.id.text1, values);
```

```
listView = (ListView) findViewById(R.id.list);
```

```
listView.setAdapter(adapter);
```

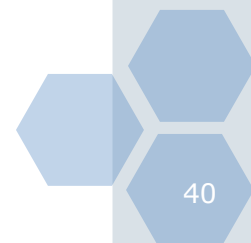




1.4.6 ListView

- Xử lý sự kiện

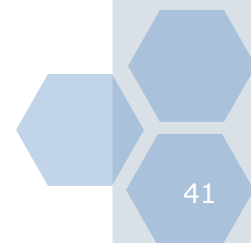
```
listView.setOnItemClickListener(new OnItemClickListener() {  
    @Override  
    public void onItemClick(  
        AdapterView<?> parent, View view, int pos, long id) {  
        String val = (String) listView.getItemAtPosition(pos);  
  
        Toast.makeText(  
            this,  
            val + "(vị trí: " + pos + ")",  
            Toast.LENGTH_LONG  
        ).show();  
    }  
});
```





1.4.7 Cách tương tác với các View

- Để tương tác được với các điều khiển (view), cần làm 2 việc:
 - Tìm đúng điều khiển cần xử lý
 - Gọi các hàm phù hợp cho điều khiển đó (chẳng hạn như thiết lập màu chữ, nội dung hiển thị, font chữ, các thuộc tính... hoặc xác định cách xử lý sự kiện)
- Tìm đúng điều khiển cần xử lý:
 - Nếu có một biến lưu trữ điều khiển cần xử lý thì bỏ qua
 - Nếu chưa có thì ta cần tìm điều khiển đó thông qua các hàm tìm kiếm của layout

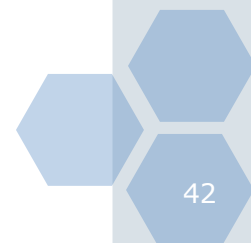




1.4.7 Cách tương tác với các View

- Khi nạp layout từ XML, mỗi view sẽ có một mã số của riêng nó (giống như CMT), mã số này là một hằng số khai báo trong thuộc tính “**android:id**”
- Hàm “**view findViewById(R.id.xyz)**” cho phép tìm đối tượng có mã số là xyz
- Sau khi tìm được view, ta chuyển kiểu view về control đúng của nó để xử lý
- Chú ý quan trọng: mỗi lần nạp lại layout sẽ xóa toàn bộ các view cũ

```
Button btnOpen = (Button) findViewById(R.id.btnOpen);  
btnOpen.setOnClickListener(new View.OnClickListener() {  
    public void onClick(View v) {  
  
        //CODE KHI NÚT OPEN ĐƯỢC NGƯỜI DÙNG CLICK  
  
    }  
});
```





❖ Bài tập: Thiết kế giao diện như sau:

a

b

c

$ax^2 + bx + c = 0$

result

vo nghiem

a

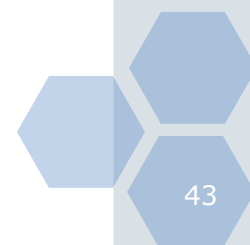
b

c

$ax^2 + bx + c = 0$

result

vo so nghiem





❖ Bài tập: Thiết kế giao diện như sau:

a

b

c

$ax^2 + bx + c = 0$

result

$x = 1$

a

b

c

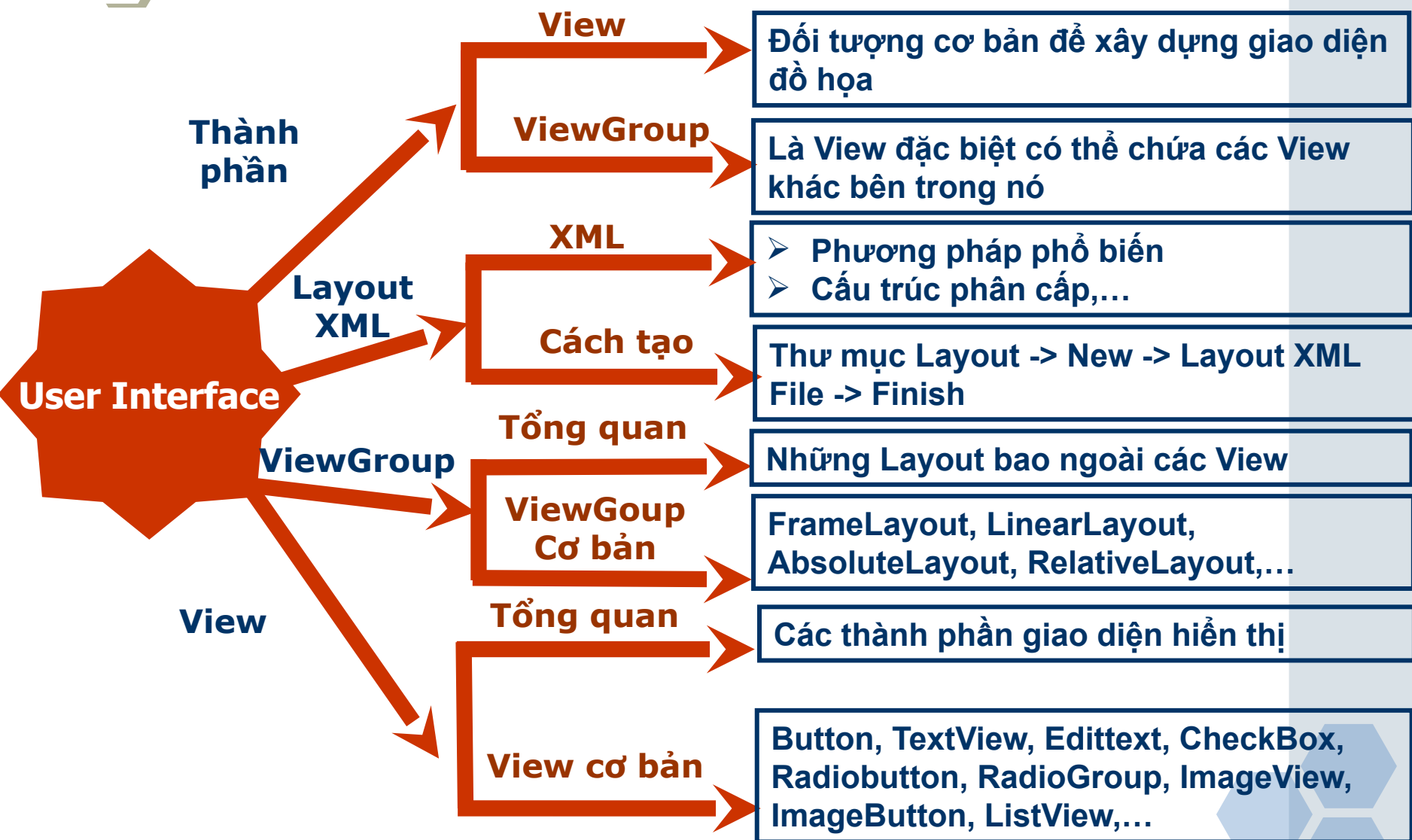
$ax^2 + bx + c = 0$

result

$x1 = 1, x2 = 2$



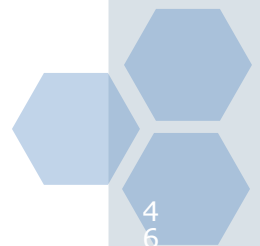
Tổng kết





Câu hỏi lý thuyết

1. View và ViewGroup là gì, khác nhau giữa View và ViewGroup
2. Nêu các tạo file layout xml. Có mấy cách để tạo giao diện xml
3. Nêu khái niệm, cách dùng và sự khác nhau giữa LinearLayout và RelativeLayout
4. Nêu khái niệm, cách dùng và sự khác nhau giữa CheckBox và Radiobutton



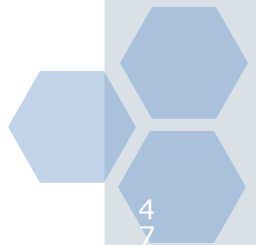


Bài tập

1. Sử dụng Table Layout để thiết kế giao diện

The screenshot shows an Android application window with a purple title bar labeled 'TableLayoutExample'. The main content area is white and contains a login form. At the top of the form is the text 'Login'. Below it are two input fields: 'User name' and 'Password'. At the bottom of the form are three buttons: 'LOGIN', 'CANCEL', and 'FORGET PASSWORD'.

- Khi bấm Cancel thì in Username và pass bằng Toast
- Khi bấm Forget Password thì mọi chữ trên Username và Password tự động biến mất
- Khi bấm nút Login thì sẽ kiểm tra Username có bằng “uneti”, password có bằng “uneti-k14” hay không.





Bài tập

2. Sử dụng Table Layout để thiết kế giao diện:

1:33

TableLayoutExample

Login

User name

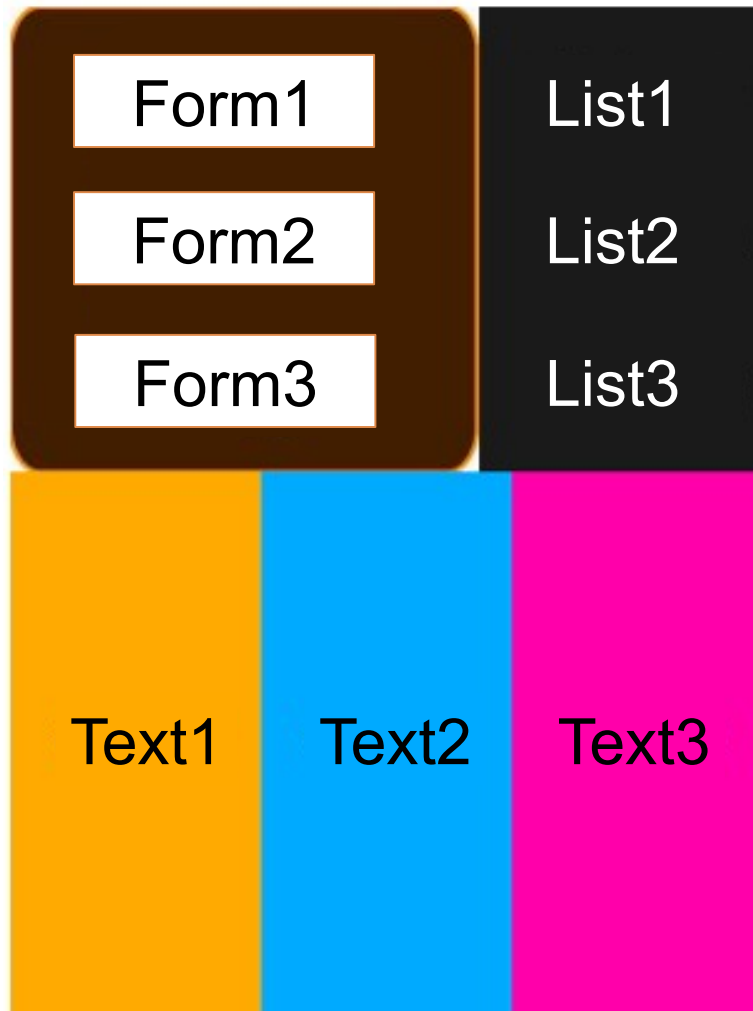
Password

LOGIN CANCEL FORGET PASSWORD

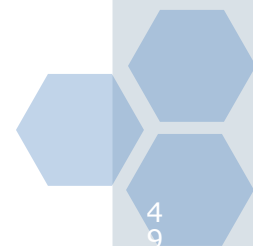
stretch stretch



3. Thiết kế Layout theo hình sau



- Form1, Form2, Form3 là EditText
- List1, List2, List3 là ListView
- Text1, Text2, Text3 là TextView
- Khi bấm List1 thì Text1 hiển thị giá trị Form1,
- Khi bấm List2 thì Text2 hiển thị giá trị Form2
- Khi bấm List3 thì Text 3 hiển thị giá trị Form3





4. Thiết kế Layout theo hình sau

1. Số nguyên từ 1 đến 100

2. Tích $12 \times 7 \times 1995$

3. Số chẵn từ 1 đến 100

4. Số lẻ từ 1 đến 100

5. Số pi

6. Tên trường UNETI

7. Số hoàn hảo từ 1 đến 100

8. 50 chữ số Fibonacci

TextViewResult

- Từ Mục 1 đến Mục 8 là ListView
- Cứ bấm một nút thì sẽ thực hiện một chức năng cụ thể và in ra TextViewResult